

Encuentro 27 — Lanzamiento del integrador

Trabajo integrador y flujo profesional

1. Metadatos de bloque

Campo	Detalle
Encuentro	27 de 36
Unidad	4 — Trabajo integrador y flujo profesional
Eje temático	4 — Trabajo integrador profesional
Carácter/Objetivo	Conceptual
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Lanzamiento del integrador
Requisitos previos	Unidad 3 finalizada (procesamiento de texto y validación); Git instalado y verificado en el equipo; repositorio grupal creado en GitHub con las carpetas <code>tp-u1/</code> , <code>tp-u2/</code> y <code>tp-u3/</code> del trabajo del año. Cada integrante tiene su fork local clonado.
Uso de celular	No permitido
Organización del trabajo	Grupos de 3-4 integrantes (los mismos de la Unidad 3). Cada grupo trabaja sobre su repositorio compartido. Se recomienda que un integrante proyecte su pantalla durante la práctica guiada para que todo el grupo vea los pasos.

Reparto de tiempos teóricos

Momento	Tiempo teórico
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

- Redactar un README de portada del repositorio grupal que describa el trabajo final, los integrantes y las tecnologías usadas.
- Descomponer el trabajo final en tareas concretas y crear issues en GitHub con asignación de responsables.

- 3. Distinguir entre un repositorio mono-rama (usado hasta ahora) y un repositorio profesional con issues y ramas.
- 4. Reconocer la consigna del trabajo final integrador como la síntesis de todos los núcleos del año.

3. Teoría mínima (20 min)

Charla rápida: El repositorio como vidriera profesional

Imaginá que tu repositorio de GitHub es la vidriera de un negocio. Hasta ahora tuviste un taller en el fondo: código funcional pero sin cartel que diga qué se vende, sin plan de tareas a la vista y con una sola puerta. En la Unidad 4 convertís esa vidriera en profesional: un README que presente el proyecto al mundo, issues que muestren el plan de trabajo y ramas que aislen cada mejora antes de mostrarla al público. No es más código; es mejor organización del código que ya sabés escribir.

Lo mínimo indispensable

README de portada. Es el archivo `README.md` en la raíz del repositorio. GitHub lo muestra automáticamente cuando alguien visita el repo. Un README profesional incluye:

Elemento	Descripción
Título del proyecto	Nombre del trabajo final (ej: "Verdulería — Sistema de venta")
Descripción breve	2-3 líneas sobre qué hace el programa
Integrantes	Lista con nombres y rol tentativo
Tecnologías	Python 3, Git, GitHub
Cómo ejecutar	<code>python main.py</code> desde la terminal
Estado del proyecto	En desarrollo / Completado (para el final)

Issues como plan de trabajo. Un issue en GitHub es una tarea con título, descripción, etiqueta y responsable. Para el trabajo final vas a crear un issue por cada módulo del programa, por ejemplo:

- #1 — README de portada y documentación inicial
- #2 — Menú principal con navegación por consola
- #3 — Registro de productos (nombre, precio, kilos en stock)
- #4 — Búsqueda de producto por nombre
- #5 — Control de venta con actualización de stock
- #6 — Gestión de factura por venta
- #7 — Protección de main y revisión final

El trabajo final integrador. Es un programa de consola en Python, dominio **verdulería**, que integra:

Núcleo del año	Cómo se refleja en el integrador
Entrada validada	<code>read_int</code> y <code>read_float</code> con <code>try/except ValueError</code> para precio y kilos
Condicionales	Validar stock suficiente antes de vender, menú con <code>if/elif/else</code>
Bucles	<code>while True</code> para el loop del menú y para reintentar entradas inválidas
Funciones	Una función por operación: <code>show_menu</code> , <code>register_product</code> , <code>sell_product</code> , <code>show_invoice</code>
Colecciones	<code>list</code> de productos, <code>dict</code> para el producto individual (nombre, precio, kilos)
Menú en memoria	Toda la información vive en variables, nada se guarda en archivos

4. Práctica guiada (35 min)

Vamos a crear el README de portada del repositorio del grupo y abrir los issues del plan de trabajo. El docente proyecta y cada grupo sigue los pasos en su propia máquina.

Paso 1: Redactar el README.md en la rama main

Desde la terminal, en la carpeta raíz del repositorio:

```
# Asegurate de estar en main y con los últimos cambios
git checkout main
git pull origin main

# Creá el README con un editor de texto o desde la terminal:
echo "# Verdulería – Sistema de venta"

> Trabajo final integrador – Programación en Python

## Descripción

Programa de consola para gestionar productos, stock y ventas de una verdulería.
Permite registrar productos por nombre, precio y kilos disponibles; realizar ventas
con descuento de stock; y emitir una factura de la venta actual.

## Integrantes

- Ana López – README y documentación
- Bruno Díaz – Menú principal y navegación
- Carla Méndez – Registro de productos y stock
- Diego Paz – Control de ventas y factura

## Tecnologías

- Python 3
- Git + GitHub

## Cómo ejecutar

Abrí la terminal y ejecutá:

python main.py
" > README.md
```

Paso 2: Abrir issues desde la web de GitHub

1. Entrá a github.com, abrí el repositorio del grupo.
2. Andá a la pestaña **Issues** → botón **New issue**.
3. Cada integrante crea al menos un issue siguiendo este modelo:

```
Título: [Módulo] Menú principal con navegación
Descripción:
  Crear un menú interactivo que muestre opciones numeradas:
  1. Registrar producto
  2. Vender producto
  3. Ver factura
  4. Salir
  El menú debe ejecutarse en un bucle hasta que el usuario elija Salir.
Asignado a: @BrunoDiaz
Labels: mejora, módulo
```

Crear los siguientes issues: - #1 — README de portada (responsable: quien redactó) - #2 — Menú principal - #3 — Registro de producto - #4 — Venta y control de stock - #5 — Factura de la venta - #6 — Protección de main y merge final

Paso 3: Verificar que los issues se ven en el repositorio

Volvé a la pestaña **Code** del repositorio: ahora el README se muestra en la página principal. Los issues deberían estar visibles en la pestaña Issues. ¡Ya tenés un repositorio profesional!

5. Ejercicio independiente (25 min)

Consigna: Cada grupo completa su README de portada y sus issues sin ayuda del docente. Usen el modelo de la práctica guiada pero personalicen:

1. **README:** agregen una sección “Estado del proyecto” donde diga “En desarrollo — Unidad 4” y una sección “Cómo contribuir” que enumere los pasos: clonar, crear rama, abrir PR.
2. **Issues:** revisen los 6 issues creados y agreguen etiquetas de prioridad (alta/media/baja) y una descripción más detallada. Cada issue debe tener al menos una tarea concreta en su descripción.

Pista: Para editar el README después de crearlo, pueden usar `echo "Nueva línea" >> README.md` para agregar contenido al final, o abrirlo en VS Code y editarlo directamente.

Solución esperada: Un README con 6 secciones completas y 6 issues con título, descripción, etiqueta y asignado. El docente verificará que cada integrante tenga al menos un issue asignado.

6. Extensión y consolidación (20 min)

Para los grupos que terminaron: sigan estos pasos para profundizar:

1. **README avanzado:** agreguen un badge de “Python 3” usando shields.io. Busquen en Google “shields.io python badge” y copien el Markdown. También pueden agregar una tabla con la estructura del proyecto:

```
tp-u1/
tp-u2/
tp-u3/
trabajo-final/
  main.py
  README.md
```

2. **Issue adicional:** creen un issue de mejora opcional que no esté en la lista base, por ejemplo: “Validar que el precio ingresado sea mayor a cero” o “Mostrar fecha y hora en la factura”.

3. **Plantilla de issue:** andá a Settings → Issues → Set up templates y creá una plantilla simple para los próximos issues del grupo.

7. Cierre (10 min)

Qué te llevás

- El README de portada es la cara de tu repositorio: sin él, el proyecto parece abandonado.
- Los issues son el plan de trabajo visible: todo el grupo sabe qué hay que hacer y quién lo hace.
- El trabajo final integrador no es código nuevo: es juntar todo lo que ya sabés en un solo programa con dominio verdulería.
- Un repositorio profesional se ve distinto a uno de práctica: tiene README, issues, y pronto ramas y PRs.

Lo que viene

En el **Encuentro 28** vas a aprender a trabajar con **ramas por feature y pull requests**: cada issue se resuelve en su propia rama, se revisa entre pares y se fusiona con main protegida. Vas a distribuir las responsabilidades del grupo por issue y a experimentar el flujo real de un equipo de desarrollo.

8. Errores comunes y trampas

1. **README vacío o con solo el título.** *Causa:* creer que el README es un trámite *Fix:* mostrar ejemplos de READMEs reales de proyectos chicos en GitHub y explicar que es lo primero que ve un reclutador.
2. **Issues sin descripción o sin asignado.** *Causa:* pensar que el título alcanza. *Fix:* establecer la regla del grupo: “sin descripción no se abre el issue”. Usar la plantilla de la práctica guiada.
3. **Un solo integrante crea todos los issues.** *Causa:* el que sabe Git hace todo. *Fix:* cada integrante debe abrir al menos un issue desde su cuenta de GitHub.
4. **Issues duplicados.** *Causa:* dos integrantes crean el mismo issue sin comunicarse. *Fix:* antes de crear, revisar los issues existentes y coordinarse en voz alta.
5. **README con errores de Markdown.** *Causa:* no conocer la sintaxis (encabezados con #, listas con -). *Fix:* mostrar una referencia rápida de Markdown y verificar con la vista previa de GitHub antes de cerrar el archivo.
6. **El README se edita pero no se versiona.** *Causa:* el grupo edita el archivo pero olvida `git add` y `git commit`. *Fix:* recordar el ritual: editar → `git add .` → `git commit -m "README: portada inicial"` → `git push`.

Encuentro 28 — Ramas y pull requests

Trabajo integrador y flujo profesional

1. Metadatos de bloque

Campo	Detalle
Encuentro	28 de 36
Unidad	4 — Trabajo integrador y flujo profesional
Eje temático	5 — Git, GitHub y trabajo colaborativo
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Ramas y pull requests
Requisitos previos	Encuentro 27 completado: README de portada creado y 6 issues abiertos en el repositorio grupal. Cada integrante tiene su cuenta de GitHub y el repositorio clonado localmente en su equipo.
Uso de celular	No permitido
Organización del trabajo	Grupos de 3-4 integrantes. Durante la práctica guiada, un integrante proyecta su pantalla; los demás siguen los pasos en su propia consola. Para el ejercicio independiente, cada grupo se divide los issues y trabaja en paralelo.

Reparto de tiempos teóricos

Momento	Tiempo teórico
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Crear una rama por feature a partir de main con `git switch -c feature/<nombre>`.
2. Publicar la rama en GitHub con `git push -u origin feature/<nombre>`.
3. Abrir un pull request en GitHub y asignar a un compañero como revisor.
4. Revisar el código de un compañero y dejar comentarios en el PR.

5. Fusionar (merge) un PR aprobado a main y configurar la protección de main en GitHub.

3. Teoría mínima (20 min)

Charla rápida: La rama como escritorio de trabajo particular

Imaginá que el grupo trabaja en una cocina profesional. `main` es el plato que se sirve al cliente: tiene que estar siempre impecable. Cada cocinero necesita su propia mesa de preparación para picar, probar y hasta equivocarse sin manchar el plato final. Esa mesa individual es tu **rama** (`feature/registro-producto`, `feature/menu-principal`, etc.). Cuando la preparación está lista, la llevás al jefe de cocina (tu revisor) para que la pruebe antes de incorporarla al plato. Ese proceso de prueba y autorización es el **pull request**. Recién cuando el revisor da el visto bueno, el plato se sirve (se fusiona a `main`).

Lo mínimo indispensable

Ramas por feature. Hasta ahora trabajaste en `main` siempre. En un proyecto profesional, cada tarea (issue) se resuelve en su propia rama. Esto permite:

- Trabajar en paralelo sin pisar el código de otro.
- Aislar cambios incompletos del código estable.
- Revisar el código antes de incorporarlo.

Comando	Qué hace
<code>git branch</code>	Lista las ramas locales; la actual tiene *
<code>git switch -c feature/algo</code>	Crea una nueva rama y se posiciona en ella
<code>git switch main</code>	Vuelve a la rama main
<code>git push -u origin feature/algo</code>	Publica la rama local en GitHub

Pull request (PR). Es una solicitud para fusionar los cambios de una rama a `main`. GitHub muestra el diff línea a línea, permite comentarios y discusión. Nadie puede saltarse la revisión: el PR se abre, se asigna un revisor, se espera aprobación y recién entonces se mergea.

Main protegida. En GitHub se configura una regla que impide pushear directo a `main`. Todo cambio entra por PR con al menos una aprobación. Así `main` siempre está estable.

4. Práctica guiada (35 min)

Vamos a crear una rama, hacer un cambio, abrir un PR y mergearlo. El docente guía paso a paso; un integrante proyecta.

Paso 1: Crear una rama desde main

```
# Posicionate en main y asegurate de tener la última versión
git switch main
git pull origin main

# Creá una rama para mejorar el README
git switch -c feature/readme-mejoras
```

Paso 2: Hacer un cambio y pushearlo a GitHub

```
# Agregá una línea al README desde la consola
echo "" >> README.md
echo "## Estado" >> README.md
echo "" >> README.md
echo "En desarrollo – Unidad 4" >> README.md

# Versioná el cambio
git add .
git commit -m "README: agregar sección Estado"

# Publicá la rama en GitHub
git push -u origin feature/readme-mejoras
```

Paso 3: Abrir un pull request en GitHub

1. Andá al repositorio en GitHub.
2. Aparece un banner que dice “feature/readme-mejoras had recent pushes” con un botón **Compare & pull request**.
3. Completá:
 - **Título:** “Agregar sección Estado al README”
 - **Descripción:** “Agrega la sección Estado con el texto ‘En desarrollo — Unidad 4’ para indicar en qué etapa está el proyecto.”
 - **Revisor:** asigná a otro integrante del grupo.
 - **Labels:** mejora.
4. Hacé clic en **Create pull request**.

Paso 4: Revisar y aprobar el PR

El integrante asignado como revisor:

1. Va a la pestaña **Pull requests** → abre el PR.
2. En la pestaña **Files changed** ve el diff línea por línea.
3. Hacé clic en **Review changes** → **Approve** → **Submit review**.
4. Opcional: dejá un comentario como “LGTM” (Looks Good To Me).

Paso 5: Merge del PR aprobado

El autor del PR (o cualquier integrante con acceso):

1. En la página del PR, hacé clic en **Merge pull request** → **Confirm merge**.
2. Opcional: hacé clic en **Delete branch** para limpiar la rama remota.

Paso 6: Actualizar main local

```
git switch main
git pull origin main
```

Verificá que el cambio está: `cat README.md` debe mostrar la nueva sección.

5. Ejercicio independiente (25 min)

Consigna: Cada grupo repite el flujo de rama-PR-merge para los 6 issues del plan, distribuyendo las responsabilidades:

1. Cada integrante toma uno o dos issues y crea su rama: `git switch -c feature/<nombre-corto>`.
2. En cada rama, hace un cambio mínimo relacionado con el issue (por ahora puede ser un archivo vacío o un esqueleto de

función).

3. Publica la rama y abre un PR asignando a otro integrante como revisor.
4. Cada integrante revisa al menos un PR de un compañero y lo aprueba.
5. Cada PR se mergea a main después de la revisión.

Pista: Si no tenés código real todavía, podés crear un archivo con un comentario. Por ejemplo, para el issue #2 “Menú principal”:

```
git switch -c feature/menu-principal
echo "# menu.py - placeholder para el menú principal" > trabajo-final/menu.py
git add .
git commit -m "menu: crear placeholder del menú principal"
git push -u origin feature/menu-principal
```

Luego abrí el PR desde GitHub.

Solución esperada: Al finalizar, el repositorio debe tener 6 PRs mergeados (uno por issue), cada uno revisado por un compañero distinto, y main con todos los placeholders de los archivos del trabajo final.

6. Extensión y consolidación (20 min)

Para los grupos que terminaron:

1. **Proteger main:** Andá a Settings → Branches → Add rule. Elegí `main` como branch name, activá “Require a pull request before merging” y “Require approvals” en 1. Guardá. Ahora nadie puede pushear directo a main.
2. **Conectar issues con PRs:** En la descripción de un PR, escribí `Closes #2` (donde 2 es el número del issue). Cuando el PR se mergea, GitHub cierra el issue automáticamente. Actualizá los PRs existentes para que cierren su issue correspondiente.
3. **Rama adicional:** Creá una rama `feature/gitignore` y agregá un archivo `.gitignore` con `__pycache__/` y `.vscode/`. Abrió PR, revisalo y mergealo.

7. Cierre (10 min)

Qué te llevás

- Las ramas aíslan tu trabajo del de tus compañeros y de main.
- `git switch -c feature/algo` es el comando clave del flujo profesional.
- El pull request es el punto de control: no se mergea código no revisado.
- La protección de main fuerza el flujo profesional: no hay atajos.
- Conectar un issue a un PR (`Closes #N`) mantiene el plan actualizado automáticamente.

Lo que viene

En el **Encuentro 29** arranca el **desarrollo del integrador**: cada grupo implementa las funciones del programa verdulería trabajando por issues y ramas, con revisiones entre pares. Vas a escribir código Python real dentro del flujo profesional que aprendiste hoy.

8. Errores comunes y trampas

1. **Olvidar `-u` en el primer push de la rama.** *Causa:* git no sabe adónde publicar. *Fix:* siempre `git push -u origin <rama>` la

primera vez; después solo `git push`.

2. **Merge conflict por editar el mismo archivo en dos ramas.** *Causa:* dos personas modifican README.md en paralelo. *Fix:* dividir archivos por integrante (uno README, otro menu.py, otro productos.py). Si el conflicto ocurre, resolverlo en GitHub o en local.
3. **Aprobar el PR sin mirar el código.** *Causa:* apuro o confianza excesiva. *Fix:* establecer la regla del grupo: “no aprobar sin leer el diff”. El revisor es responsable de lo que se mergea.
4. **PR sin descripción.** *Causa:* pensar que el título basta. *Fix:* usar la descripción para explicar qué cambia y por qué.
5. **Hacer el merge sin esperar la revisión.** *Causa:* querer terminarlo rápido. *Fix:* con la protección de main activada no es posible; si no está activada, el grupo debe acordar la disciplina.
6. **Mezclar cambios de otro issue en la misma rama.** *Causa:* un integrante trabaja dos tareas en la misma rama para ahorrar tiempo. *Fix:* cada rama resuelve un solo issue. Si se mezclan, separar con `git cherry-pick` o crear una rama nueva desde main.

Encuentro 29 — Desarrollo del integrador

Trabajo integrador y flujo profesional

1. Metadatos de bloque

Campo	Detalle
Encuentro	29 de 36
Unidad	4 — Trabajo integrador y flujo profesional
Eje temático	4 — Trabajo integrador profesional
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Desarrollo del integrador
Requisitos previos	Encuentro 28 completado: main protegida, ramas por issue creadas y al menos un PR mergeado. Cada grupo tiene el esqueleto del trabajo final con los placeholders de archivos.
Uso de celular	No permitido
Organización del trabajo	Grupos de 3-4 integrantes. Cada integrante trabaja en su rama asignada (issue) y abre PR cuando su función está lista. El docente circula entre los grupos para destrabar PRs y resolver conflictos.

Reparto de tiempos teóricos

Momento	Tiempo teórico
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Implementar el módulo asignado del programa verdulería en su rama feature.
2. Abrir un pull request con código funcional (no placeholders) y asignar revisor.
3. Revisar el código de un compañero contra la checklist de defectos frecuentes de Python.
4. Resolver conflictos de merge simples en GitHub o local.
5. Integrar el módulo revisado a main y verificar que el programa sigue funcionando.

3. Teoría mínima (20 min)

Charla rápida: La verdulería como sistema integrado

Imaginá una verdulería real. El dueño necesita: saber qué productos tiene y cuánto pesa cada uno (stock), poder registrar un producto nuevo cuando llega el camión, cobrar una venta restando lo que se vendió del stock y darle un ticket al cliente. Esas cuatro operaciones son tu programa. Cada integrante construye una pieza (registro, venta, factura, menú) y todas se conectan a través del menú principal. La magia no está en cada pieza por separado, está en que funcionan juntas compartiendo los datos en memoria.

Lo mínimo indispensable

Estructura del programa integrador (dominio verdulería).

```
trabajo-final/  
main.py          # Menú principal que conecta todo  
productos.py     # Registro y consulta de productos  
ventas.py        # Control de venta y actualización de stock  
factura.py       # Generación de factura de la venta actual
```

Cada archivo contiene funciones que trabajan sobre listas y diccionarios compartidos. El estado se pasa entre funciones como parámetros. No hay archivos ni persistencia.

Funciones canónicas para el integrador:

```
def read_int(prompt):  
    # Lee un entero con validación: input() devuelve str,  
    # int() lanza ValueError si no es numérico (spike 9.1).  
    while True:  
        try:  
            return int(input(prompt))  
        except ValueError:  
            print("Eso no es un número entero; intenta de nuevo.")  
  
def read_float(prompt):  
    # Lee un decimal con validación: float() lanza ValueError  
    # si el texto no puede convertirse (spike 9.2).  
    while True:  
        try:  
            return float(input(prompt))  
        except ValueError:  
            print("Eso no es un número válido; intenta de nuevo.")  
  
def show_menu(options):  
    # Muestra un menú numerado. options es una lista de strings.  
    # Ejemplo: ["Registrar producto", "Vender", "Factura", "Salir"]  
    print("\n=== VERDULERÍA ===")  
    for i, option in enumerate(options, start=1):  
        print(f"{i}. {option}")  
    print()
```

Reglas del flujo de trabajo de hoy: 1. Cada integrante trabaja en su rama: `git switch feature/<issue>`. 2. Cuando la función está lista (ejecutable, probada), abrí PR. 3. El revisor verifica el código contra la checklist de defectos frecuentes. 4. Merge solo después de aprobación. 5. Después del merge, cada integrante actualiza su main: `git switch main && git pull`.

4. Práctica guiada (35 min)

Vamos a implementar el módulo de registro de productos completo, paso a paso.

Paso 1: Ubicarse en la rama correcta

```
git switch main  
git pull origin main  
git switch -c feature/registro-productos
```

Paso 2: Escribir el archivo `productos.py` con la función de registro

```

# productos.py
# Funciones para registrar y consultar productos en una verdulería.
# Cada producto es un dict con nombre, precio por kilo y kilos en stock.

def register_product(product_list):
    """Registra un nuevo producto: pide nombre, precio y kilos.

    Recibe La lista de productos y agrega un nuevo dict.
    No devuelve nada (None): muta La lista recibida.
    """
    print("\n--- Registrar producto ---")
    name = input("Nombre del producto: ").strip().lower()

    # Verificar si el producto ya existe
    for product in product_list:
        if product["name"] == name:
            print(f"El producto '{name}' ya está registrado.")
            return

    price = read_float("Precio por kilo ($): ")
    kilos = read_float("Kilos en stock: ")

    product_list.append({
        "name": name,
        "price": price,
        "kilos": kilos
    })
    print(f"✓ '{name.capitalize()}' registrado con ${price:.2f}/kg y {kilos} kg.")

def list_products(product_list):
    """Muestra todos los productos con su precio y stock."""
    if not product_list:
        print("No hay productos registrados.")
        return

    print("\n--- Productos disponibles ---")
    for product in product_list:
        print(f"{product['name'].capitalize():15} "
              f"${product['price']:.2f}/kg "
              f"{product['kilos']:.2f} kg")
    print(f"Total: {len(product_list)} producto(s).")

if __name__ == "__main__":
    # Prueba rápida de Las funciones
    test_list = [
        {"name": "manzana", "price": 2.5, "kilos": 10},
        {"name": "banana", "price": 1.8, "kilos": 15},
    ]
    list_products(test_list)
    register_product(test_list)
    list_products(test_list)

```

Paso 3: Probar el módulo

```
python productos.py
```

Salida esperada:

```

--- Productos disponibles ---
Manzana      $2.50/kg  10.00 kg
Banana       $1.80/kg  15.00 kg
Total: 2 producto(s).

--- Registrar producto ---
Nombre del producto: Pera
Precio por kilo ($): 3.2
Kilos en stock: 8
✓ 'Pera' registrado con $3.20/kg y 8.00 kg.

--- Productos disponibles ---
Manzana      $2.50/kg  10.00 kg
Banana       $1.80/kg  15.00 kg
Pera         $3.20/kg  8.00 kg
Total: 3 producto(s).

```

Paso 4: Publicar, abrir PR y asignar revisor

```

git add .
git commit -m "productos: implementar registro y listado"
git push -u origin feature/registro-productos

```

Andá a GitHub, abrí el PR con título “Implementar módulo de registro de productos” y asigná un revisor.

Paso 5: Revisar el PR de un compañero

El revisor mira el diff y verifica la checklist de defectos frecuentes:

- ¿Las funciones usan `return`? ✓
- ¿No hay `lista = lista.append()`? ✓
- ¿Los mensajes al usuario usan f-strings? ✓
- ¿La entrada se valida con `try/except ValueError`? ✓
- ¿No hay código suelto fuera del bloque `if __name__ == "__main__":`? ✓

Si todo está bien, **Approve** y **Merge**.

5. Ejercicio independiente (25 min)

Consigna: Cada integrante implementa su módulo asignado siguiendo el modelo de la práctica guiada:

Issue	Archivo	Qué implementar
#2 Menú principal	<code>main.py</code>	Función <code>show_menu()</code> que recibe una lista de opciones y el loop principal del programa
#3 Registro de producto	<code>productos.py</code>	Completar: ya tienen la base de la práctica, agregar función <code>find_product()</code> por nombre
#4 Venta y control de stock	<code>ventas.py</code>	Función <code>sell_product()</code> que descuenta kilos y valida stock suficiente
#5 Factura de la venta	<code>factura.py</code>	Función <code>show_invoice()</code> que muestra los items vendidos y el total de la venta actual

Cada módulo se prueba con `python <archivo>.py` antes de abrir PR.

Pista: La función `find_product()` recibe el nombre y la lista, recorre con un bucle `for` y devuelve el producto (dict) si existe o `None` si no.

Solución esperada: Al final del encuentro, cada integrante debe tener al menos un PR mergeado con código funcional de su módulo. El programa completo aún no corre de forma integrada (falta unirlos en main.py), pero cada pieza funciona por separado.

6. Extensión y consolidación (20 min)

Para los grupos que terminaron:

1. **Conectar módulos:** En `main.py`, importá las funciones de los otros módulos y armá el menú completo que llame a las funciones correctas. Probá el integrador completo con `python main.py`.
2. **Función adicional:** Agregá una función `update_price(product_list)` que permita cambiar el precio de un producto existente. Creá un issue nuevo (#7) y trabajalo en su rama.
3. **README dinámico:** Actualizá el README con la estructura real del proyecto y los nombres de los módulos.

7. Cierre (10 min)

Qué te llevás

- Cada módulo se implementa en su rama y se integra vía PR: así no se rompe lo que ya funciona.
- Las funciones de validación (`read_int`, `read_float`) se reusan en todos los módulos: no las copies, ponelas en un archivo compartido o pasalas como parámetro.
- Revisar el código del compañero con la checklist de defectos frecuentes te hace mejor programador: aprendés viendo errores ajenos.
- El programa integrador empieza a tomar forma: ya tenés productos, ventas y factura como piezas independientes.

Lo que viene

En el **Encuentro 30** vas a **consolidar el integrador**: cerrar issues pendientes, integrar todos los módulos en un solo programa ejecutable, hacer los ajustes finales y empezar a preparar la defensa individual. Es la última clase de desarrollo antes del cierre.

8. Errores comunes y trampas

1. **Función sin `return` que se usa como valor.** *Causa:* `register_product()` muta la lista pero no devuelve nada; si hacés `resultado = register_product(...)`, `resultado` es `None`. *Fix:* estas funciones trabajan por efecto y no necesitan return.
2. **No validar entrada en precio o kilos.** *Causa:* usar `float(input(...))` sin `try/except`. *Fix:* siempre `read_float()` con el patrón de validación canónico.
3. **Comparar strings sin normalizar.** *Causa:* el usuario escribe "Manzana" y el programa busca "manzana". *Fix:* aplicar `.strip().lower()` al leer el nombre.
4. **PR con código que no compila.** *Causa:* hacer commit de un archivo a mitad de edición. *Fix:* antes de abrir el PR, probar `python archivo.py`. Si falla, no se abre el PR.
5. **Conflictos por archivos duplicados.** *Causa:* dos integrantes crean `main.py` en distintas ramas. *Fix:* acordar qué archivos crea cada uno y no tocarlos hasta que el primer PR se mergee.
6. **Olvidar pushear el commit antes del PR.** *Causa:* el commit está local pero no en GitHub; el PR aparece con 0 commits. *Fix:* siempre verificar con `git push` antes de abrir el PR.

Encuentro 30 — Consolidación del integrador

Trabajo integrador y flujo profesional

1. Metadatos de bloque

Campo	Detalle
Encuentro	30 de 36
Unidad	4 — Trabajo integrador y flujo profesional
Eje temático	4 — Trabajo integrador profesional
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Consolidación del integrador
Requisitos previos	Encuentro 29 completado: cada integrante implementó y mergeó su módulo (productos, ventas, factura, menú). El repositorio tiene código funcional en ramas feature mergeadas a main.
Uso de celular	No permitido
Organización del trabajo	Grupos de 3-4 integrantes. Los grupos que completaron todos los módulos trabajan en la integración y preparación de la defensa. Los grupos con módulos pendientes priorizan cerrarlos al inicio. El docente atiende a los grupos más atrasados.

Reparto de tiempos teóricos

Momento	Tiempo teórico
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Integrar todos los módulos del programa en un solo archivo `main.py` ejecutable.
2. Probar el programa completo (ruta feliz, errores de entrada, opción inválida).
3. Resolver los issues pendientes y cerrarlos con PRs.
4. Preparar la estructura de la defensa individual: qué va a explicar cada integrante.

5. Hacer el commit de cierre del encuentro con mensaje significativo.

3. Teoría mínima (20 min)

Charla rápida: El ensayo general antes del estreno

Imaginá que tu grupo es una banda que ensayó cada tema por separado. Hoy es el ensayo general: todo el grupo toca junto, los temas suenan en el orden correcto, se afinan los volúmenes y se prueban los equipos. Si algo suena mal, se ajusta antes del concierto (que es la defensa). El público no escucha los ensayos individuales: escucha el resultado final. Lo mismo con el integrador: no importa qué tan bien funcione cada módulo por separado si no funcionan juntos.

Lo mínimo indispensable

Integración de módulos. Cada módulo (`productos.py`, `ventas.py`, `factura.py`) expone funciones que se importan en `main.py` y se conectan a través del menú.

Para que `main.py` pueda usar las funciones de los otros archivos, se importan al principio:

```
from productos import register_product, list_products, find_product
from ventas import sell_product
from factura import show_invoice
```

El estado compartido (la lista de productos y la factura actual) vive en `main.py` y se pasa como argumento a cada función.

Ciclo de vida del programa: 1. Inicializa lista de productos vacía y factura vacía. 2. Muestra el menú y espera opción. 3. Según la opción, llama a la función correspondiente. 4. Vuelve al menú hasta que el usuario elige “Salir”. 5. Muestra mensaje de despedida y termina.

Checklist de prueba canónica: 1. Ruta feliz: registrar producto, vender, ver factura, salir. 2. Entrada no numérica donde se espera número. 3. Lista vacía: factura sin productos, lista sin productos. 4. Opción de menú inexistente.

Preparación de la defensa. Cada integrante debe poder explicar:

- **Su módulo:** qué hace, cómo lo implementó, qué validaciones tiene.
- **Un error que encontró y corrigió:** muestra aprendizaje.
- **El flujo del programa:** cómo se conecta su módulo con los demás.
- **Un PR que revisó:** qué observó y por qué aprobó.

4. Práctica guiada (35 min)

Vamos a integrar todos los módulos en un solo programa ejecutable.

Paso 1: Posicionarse en main y crear rama de integración

```
git switch main
git pull origin main
git switch -c feature/integracion-final
```

Paso 2: Escribir `main.py` que importe y conecte todo

```

# main.py
# Programa integrador - Verdulería
# Integra registro de productos, venta con stock y facturación.
# Todos los datos viven en memoria (Listas y dicts).

from productos import register_product, list_products, find_product
from ventas import sell_product
from factura import show_invoice

def read_int(prompt):
    # Lee un entero con validación (spike 9.1).
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Eso no es un número entero; intenta de nuevo.")

def main():
    # Lista compartida: cada producto es un dict
    # {"name": str, "price": float, "kilos": float}
    product_list = [
        {"name": "manzana", "price": 2.50, "kilos": 10.0},
        {"name": "banana", "price": 1.80, "kilos": 15.0},
        {"name": "naranja", "price": 1.20, "kilos": 20.0},
    ]
    # Factura de la venta actual: lista de items vendidos
    current_invoice = []

    options = ["Registrar producto",
               "Listar productos",
               "Vender producto",
               "Ver factura actual",
               "Salir"]

    while True:
        print("\n=== VERDULERÍA - Sistema de venta ===")
        for i, option in enumerate(options, start=1):
            print(f"{i}. {option}")

        choice = read_int("Opción: ")

        if choice == 1:
            register_product(product_list)
        elif choice == 2:
            list_products(product_list)
        elif choice == 3:
            sell_product(product_list, current_invoice)
        elif choice == 4:
            show_invoice(current_invoice)
        elif choice == 5:
            print("Gracias por usar Verdulería. ¡Hasta la próxima!")
            break
        else:
            print("Opción inválida. Elegí un número del 1 al 5.")

if __name__ == "__main__":
    main()

```

Paso 3: Verificar que `productos.py` tiene `find_product()`

Si no la tiene, agrega:

```
def find_product(product_list, name):
    """Busca un producto por nombre (normalizado). Devuelve el dict o None."""
    for product in product_list:
        if product["name"] == name.strip().lower():
            return product
    return None
```

Paso 4: Verificar que `ventas.py` existe con `sell_product()`

```
# ventas.py
# Control de venta: descuento stock y agrega items a la factura.

def sell_product(product_list, invoice):
    """Registra una venta: busca producto, valida stock, descuenta y factura."""
    print("\n--- Vender producto ---")
    name = input("Nombre del producto: ").strip().lower()
    product = find_product(product_list, name)

    if product is None:
        print(f"Producto '{name}' no encontrado.")
        return

    print(f"{product['name'].capitalize()} - "
          f"${product['price']:.2f}/kg - {product['kilos']:.2f} kg disponibles")

    kilos = read_float("Kilos a vender: ")

    if kilos <= 0:
        print("La cantidad debe ser mayor a cero.")
        return

    if kilos > product["kilos"]:
        print(f"Stock insuficiente. Hay {product['kilos']:.2f} kg disponibles.")
        return

    # Descontar del stock
    product["kilos"] -= kilos

    # Agregar a la factura
    item_total = kilos * product["price"]
    invoice.append({
        "name": product["name"],
        "kilos": kilos,
        "price": product["price"],
        "total": item_total
    })
    print(f"✓ Venta registrada: {kilos:.2f} kg de "
          f"{product['name'].capitalize()} - ${item_total:.2f}")

def read_float(prompt):
    # Lee un decimal con validación (spike 9.2).
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Eso no es un número válido; intenta de nuevo.")
```

Paso 5: Ejecutar el programa completo

```
python main.py
```

Probar: 1 (registrar Pera, 3.5/kg, 8kg), 3 (vender 2kg de manzana), 4 (ver factura), 5 (salir).

Paso 6: Si funciona, commitear y abrir PR

```
git add .
git commit -m "integrador: integrar todos los módulos en main.py"
git push -u origin feature/integracion-final
```

Abrir PR, revisión entre pares, mergear.

5. Ejercicio independiente (25 min)

Consigna: Cada grupo termina el integrador siguiendo estos pasos:

1. **Cerrar issues pendientes:** revisá qué issues están abiertos y asegurate de que todos tengan PR mergeado. Si un issue no está resuelto, asignalo a un integrante ahora.
2. **Completar `factura.py`** si falta: debe mostrar cada ítem (nombre, kilos, precio, subtotal) y el total de la factura.
3. **Probar el programa completo** con la checklist canónica:
 - ☐ Registrar producto y verlo en la lista.
 - ☐ Vender producto con stock suficiente.
 - ☐ Vender producto sin stock suficiente (debe mostrar mensaje de error).
 - ☐ Vender producto que no existe.
 - ☐ Ver factura sin haber vendido nada.
 - ☐ Ingresar texto donde se espera número.
 - ☐ Elegir opción 6 (inexistente).
 - ☐ Salir del programa.
4. **Preparar la defensa:** cada integrante escribe 3-4 líneas sobre qué va a explicar en la defensa individual. Guardarlo como `DEFENSA.md` en el repo.

Pista: Para la factura, si la lista `invoice` está vacía mostrará "No hay ventas registradas."

Solución esperada: Un programa que ejecuta las 8 pruebas de la checklist sin errores. Archivo `DEFENSA.md` en la raíz. Todos los issues cerrados.

6. Extensión y consolidación (20 min)

Para los grupos que terminaron:

1. **Mejora opcional:** Agregá una opción "Actualizar precio" que pida el nombre del producto y el nuevo precio, y lo actualice en la lista. Creá un issue, rama, PR y mergealo.
2. **Refactor:** Pasá `read_int` y `read_float` a un archivo `utils.py` e importalas desde cada módulo. Creá el PR correspondiente.
3. **Verificación final del repositorio:** Revisá que el README tenga las instrucciones correctas de ejecución y que `.gitignore` exista con `__pycache__`.

7. Cierre (10 min)

Qué te llevás

- Integrar módulos en Python es cuestión de `from ... import ...` en la parte superior del archivo.
- El estado compartido (lista de productos, factura) se pasa como argumento: cada función recibe lo que necesita.
- La prueba canónica no es opcional: sin ella, el programa puede fallar en la defensa.
- Preparar la defensa ahora evita el estrés de último minuto: sabés qué decir y cómo mostrarlo.

Lo que viene

En el **Encuentro 31** es el **cierre de la Unidad 4 y entrega final**: sistematizamos todo el trabajo, hacemos el commit final del trabajo integrador y preparamos la defensa individual delante del docente. Después del encuentro 31 viene el **Encuentro 32 — Defensa del integrador**, donde cada integrante presenta su trabajo individualmente.

8. Errores comunes y trampas

1. **Import circular o archivo que no existe.** *Causa:* `main.py` importa `productos` pero `productos.py` está en otra carpeta o no existe. *Fix:* verificar que todos los `.py` estén en `trabajo-final/` y que los imports no tengan puntos.
2. **La factura se borra al volver al menú.** *Causa:* en lugar de acumular ítems, la función crea una factura nueva cada vez. *Fix:* pasar la misma lista `current_invoice` a todas las llamadas de `sell_product()`.
3. **Menú que no vuelve después de una opción.** *Causa:* faltó indentar o el `break` corta el loop. *Fix:* verificar que la opción 5 (Salir) sea la única que interrumpe el `while True`.
4. **El programa crashea si se ingresa un float en `read_int`.** *Causa:* `int()` no acepta strings con punto decimal (spike 9.1). *Fix:* usar `read_float()` para precio y kilos; `read_int()` solo para opciones del menú y cantidades enteras.
5. **El producto se busca exacto sin normalizar.** *Causa:* el usuario escribe “Manzana” y el código busca “manzana”. *Fix:* aplicar `.strip().lower()` al nombre de búsqueda y al momento de registrar.
6. **Commit de integración sin probar.** *Causa:* mergear el PR de integración sin ejecutar `python main.py`. *Fix:* probar antes de mergear; si falla, no se mergea.

Encuentro 31 — Cierre U4: entrega final

Trabajo integrador y flujo profesional · Encuentro de cierre de unidad

1. Metadatos de bloque

Campo	Detalle
Encuentro	31 de 36
Unidad	4 — Trabajo integrador y flujo profesional
Eje temático	4 — Trabajo integrador profesional
Carácter/Objetivo	Actitudinal
Estructura	cierre
Duración teórica	120 minutos (2 horas reloj)
TP obligatorio	Trabajo final: programa integrador con repositorio profesional
Concepto nuevo	Cierre U4: entrega final
Requisitos previos	Encuentro 30 completado: programa integrador funcionando con todos los módulos, todos los issues cerrados, DEFENSA.md con el plan de cada integrante. Repositorio con main protegida y ramas feature mergeadas.
Uso de celular	No permitido
Organización del trabajo	Grupos de 3-4 integrantes. Primero sesión grupal de cierre y sistematización; luego cada integrante prepara su defensa individual. El docente pasa grupo por grupo para la verificación final del repositorio.

Reparto de tiempos teóricos (plantilla de cierre de unidad)

Momento	Tiempo teórico
Apertura	10 min
Consolidación	40 min
Trabajo del TP	45 min
Ciclo de entrega	15 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Sistematizar el recorrido de la Unidad 4: desde el README y los issues hasta el programa integrador funcionando.
2. Verificar que el repositorio cumple con todos los requisitos del TP (README, issues, PRs, main protegida, programa ejecutable).
3. Preparar la exposición individual de la defensa: qué hace cada parte del programa y cómo se trabajó en equipo.

4. Realizar el commit y etiqueta final de entrega del trabajo integrador.
5. Reconocer los núcleos del año integrados en el programa final (entrada validada, condicionales, bucles, funciones, colecciones, menú en memoria).

3. Apertura y puente (10 min)

Charla rápida: De las piezas sueltas al programa que funciona

Al principio del año programabas líneas sueltas en la consola. Luego aprendiste condicionales, bucles, funciones, listas y diccionarios. En la Unidad 3 empezaste a validar entrada como un profesional. Y en esta unidad no aprendiste nuevas instrucciones de Python: aprendiste a organizar tu trabajo con Git y GitHub como se hace en la industria. El trabajo final no es un ejercicio más: es la prueba de que podés construir un programa completo, solo o en equipo, desde cero, con código limpio y un repositorio profesional. Mañana en la defensa no mostrás código: mostrás que pensás como programador.

Lo mínimo indispensable

¿Qué entregás hoy? El TP “Programa integrador con repositorio profesional” consiste en:

Requisito	Cómo se verifica
README de portada	Se ve en la raíz del repositorio en GitHub
Issues con plan de trabajo	Pestaña Issues: todos cerrados o con PR mergeado
Ramas por feature	Pestaña Branches: deben existir las ramas feature
Pull requests con revisión	Pestaña Pull requests: al menos un PR por integrante revisado
Main protegida	Settings → Branches: regla activa
Programa ejecutable	<code>python main.py</code> funciona en el equipo del docente
Sin persistencia	No hay <code>.csv</code> , <code>.json</code> , <code>open()</code> en el código
DEFENSA.md	Archivo con el plan de cada integrante

¿Qué viene después (Encuentro 32)? La defensa del integrador es individual. Cada integrante se sienta con el docente, abre su repositorio, explica el programa, muestra el flujo de trabajo, ejecuta el programa y responde preguntas. Duración estimada: 10-15 minutos por integrante.

4. Consolidación (40 min)

Actividad de repaso de núcleos. Cada grupo revisa su programa integrador y completa la siguiente tabla identificando dónde se aplica cada núcleo del año:

Núcleo	¿Dónde aparece en nuestro programa?	Archivo
Entrada validada	<code>read_int()</code> y <code>read_float()</code> con <code>try/except ValueError</code>	<code>main.py</code>
Condicionales	<code>if choice == 1:</code> en el menú; <code>if kilos > product["kilos"]</code> en ventas	<code>main.py</code> , <code>ventas.py</code>
Bucles	<code>while True</code> del menú; <code>while True</code> de validación	<code>main.py</code>
Funciones	<code>register_product()</code> , <code>sell_product()</code> , <code>show_invoice()</code> , <code>find_product()</code>	todos los archivos
Colecciones	<code>list</code> de productos, <code>dict</code> por producto, <code>list</code> de factura	<code>main.py</code> , <code>productos.py</code> , <code>factura.py</code>
Menú en memoria	Sin archivos ni BD: todo en listas y dicts	<code>main.py</code>

Repaso del flujo profesional de Git. Cada grupo verifica en su repositorio:

1. Que el README tenga las 6 secciones del modelo.
2. Que haya al menos 5 issues cerrados con PR mergeados.
3. Que cada integrante haya creado al menos una rama y un PR.
4. Que `main` esté protegida (Settings → Branches).
5. Que exista `.gitignore` con `__pycache__/`.

Repaso de la defensa. Cada integrante practica en voz baja su explicación (3-5 minutos):

1. **Apertura:** “Este es el repositorio del grupo. Yo trabajé en el módulo de XX.”
2. **Demo del programa:** Ejecutar `python main.py` , mostrar 2-3 operaciones.
3. **Recorrido por issues y PRs:** “Acá está el issue #3 que me asignaron, y acá mi PR con la revisión de mi compañero.”
4. **Aprendizaje:** “El error más importante que corregí fue XX.”
5. **Cierre:** “Lo que más me costó fue XX y lo resolví haciendo XX.”

Cierre del repositorio. Hacer el commit final de la unidad:

```
git switch main
git pull origin main
git add .
git commit -m "trabajo-final: cierre de la unidad 4 – entrega del integrador"
git push origin main
```

Opcional: crear un tag de versión.

```
git tag -a v1.0 -m "Entrega del trabajo final integrador – U4"
git push origin v1.0
```

5. Trabajo del TP (45 min)

Consigna final: Cada grupo prepara su repositorio para la defensa y resuelve los últimos detalles.

Paso 1: Verificar el programa completo

Ejecutar `python main.py` y probar:

1. Registrar “papa” (1.2, 25 kg).
2. Listar productos: debe aparecer papa al final.
3. Vender 3 kg de papa.
4. Ver factura: debe mostrar papa 3 kg × \$1.20 = \$3.60.
5. Intentar vender 100 kg de banana (stock: 15 kg) → “Stock insuficiente”.
6. Vender 5 kg de banana.
7. Ver factura: debe tener papa y banana.
8. Elegir opción inválida (6) → “Opción inválida”.
9. Ingresar letra donde va número → mensaje de validación.
10. Salir.

Paso 2: Verificar el repositorio

Abrir GitHub y revisar:

- ☐ README.md en la raíz visible y completo.
- ☐ Issues: todos cerrados o en estado Done.
- ☐ Pull requests: al menos 3 mergeados (uno por integrante).
- ☐ Rama `main` protegida (Settings → Branches).
- ☐ `.gitignore` con `__pycache__` y `.vscode/`.
- ☐ `DEFENSA.md` con el plan de cada integrante.
- ☐ Último commit: mensaje de cierre.

Paso 3: Preparar la defensa individual

Cada integrante edita `DEFENSA.md` con su sección personal:

```
# Defensa del trabajo final integrador

## Integrante: Ana López

### Módulo a cargo
Registro de productos (productos.py): funciones register_product(), list_products(),
find_product(). Validación de precio y kilos con read_float().

### Error corregido
Olvidé normalizar el nombre con .strip().lower(), entonces "Manzana" no coincidía
con "manzana" en la búsqueda. Lo corregí en todas las funciones de productos.py.

### PR revisado
Revisé el PR de Bruno (#4 – ventas.py). Verifiqué que validara stock suficiente
antes de descontar y que la factura acumulara items, no los reemplazara.

### Pregunta preparada
¿Qué pasa si dos integrantes modifican el mismo archivo en distintas ramas?
Respuesta: se produce un conflicto de merge. GitHub lo marca y hay que resolverlo
manualmente eligiendo qué cambios conservar.
```

Pista: En `DEFENSA.md`, cada integrante escribe su sección debajo de un encabezado con su nombre. No borren las secciones de los demás.

Solución esperada: Repositorio completo con todos los checkboxes marcados y `DEFENSA.md` con una sección por integrante.

6. Ciclo de entrega (15 min)

Verificación del docente. El docente pasa grupo por grupo y verifica:

1. `python main.py` funciona sin errores.
2. El repositorio de GitHub está actualizado con el último commit.
3. `DEFENSA.md` existe con secciones para todos los integrantes.
4. Cada integrante sabe qué va a decir en la defensa.

Feedback grupal. El docente comenta:

- **Aciertos:** qué funcionó bien en el trabajo del grupo.
- **Áreas de mejora:** qué podrían hacer distinto en un próximo proyecto.
- **Nota del TP:** confirmación de la nota grupal (aprobado/desaprobado).

Cierre administrativo. Cada grupo confirma que el repositorio está en condiciones de ser evaluado. Si falta algo, se anota como deuda técnica para resolver antes del Encuentro 32.

7. Cierre (10 min)

Qué te llevás

- Un programa integrador que funciona y que escribiste vos: entrada validada, condicionales, bucles, funciones, listas y diccionarios, todo en un solo programa de consola.
- Un repositorio profesional: README, issues, ramas, PRs revisados y main protegida.
- El flujo de trabajo real de un equipo de desarrollo: cada tarea en su rama, revisión entre pares, integración continua.
- La experiencia de preparar y ensayar una defensa técnica: explicar tu código, justificar decisiones y mostrar resultados.

Lo que viene

En el **Encuentro 32 — Defensa del integrador** vas a presentar tu trabajo final individualmente frente al docente. Vas a mostrar tu repositorio, ejecutar el programa, explicar tu módulo y responder preguntas sobre el código y el flujo de trabajo. Es la culminación del curso: demostrar que podés programar y trabajar en equipo como un profesional.

8. Errores comunes y trampas

1. **DEFENSA.md incompleto o con una sola sección.** *Causa:* un integrante escribe y los demás no contribuyen. *Fix:* cada integrante edita su propia sección y hace commit en su rama; después se mergea.
2. **Repositorio desactualizado local vs remoto.** *Causa:* hicieron commit local pero olvidaron `git push`. *Fix:* antes del ciclo de entrega, verificar en GitHub que el último commit esté visible.
3. **Programa que funciona en una máquina pero no en otra.** *Causa:* diferencia de versión de Python, archivos faltantes, rutas absolutas. *Fix:* probar `python main.py` en la máquina del docente o pasarle el repositorio clonado a otro grupo para que lo pruebe.
4. **No saber qué decir en la defensa.** *Causa:* no prepararon la explicación de su módulo. *Fix:* practicar 3 veces la explicación con el cronómetro antes del Encuentro 32.
5. **El docente encuentra un error que no vieron.** *Causa:* se saltaron la prueba de casos borde (factura vacía, stock insuficiente, opción inválida). *Fix:* correr la checklist completa antes de la entrega.
6. **No recordar el flujo de Git durante la defensa.** *Causa:* confundir `git pull` con `git push`, o no poder explicar para qué sirve una rama. *Fix:* en `DEFENSA.md`, cada integrante incluye una línea con el comando que más usó.

Evaluación de la Unidad 4 — Encuentro 32

Evaluación de la instancia «Evaluación de la Unidad 4 — Encuentro 32» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-u4-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-u4-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación de la Unidad 4 — Encuentro 32
Unidad evaluada	4 — Trabajo integrador y flujo profesional
Eje temático	4 — Trabajo integrador profesional
Carácter/Objetivo	Defensa individual del trabajo integrador + verificación práctica de modificación guiada sobre el propio programa del grupo.
Destinatarios	Todo el curso
Duración teórica	120 minutos (2 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-u4.md</code> · <code>evaluacion-u4-version-a.md</code> · <code>evaluacion-u4-version-a-anexo-docente.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (120 min)

Momento	Tiempo	Qué ocurre
Apertura y organización	10 min	El docente explica la dinámica: defensa individual (15 min c/u) + ventana de verificación práctica. Se asigna orden de defensa.
Defensa individual (rotación)	60 min	Cada estudiante se sienta con el docente, muestra su repositorio, ejecuta el programa y responde preguntas. 10-15 min por persona.
Verificación práctica (ventana)	40 min	Mientras esperan su turno o después de defender, los estudiantes realizan la modificación guiada sobre su propio código.
Cierre	10 min	Entrega de la modificación por Git. El docente anuncia la devolución para el encuentro siguiente.
Total	120 min	

3. Regla canónica de la instancia

La entrega del trabajo final se verificó en el encuentro 31. En este encuentro 32 se realiza la defensa individual y una

verificación práctica breve de modificación guiada. Devolución al inicio del encuentro 33.

4. Defensa individual del TP (modalidad)

Cada integrante se sienta individualmente con el docente (10-15 min). Durante la defensa:

1. **Apertura (1 min):** el estudiante muestra el repositorio en GitHub.
2. **Demo del programa (3 min):** ejecuta `python main.py` y recorre las opciones principales.
3. **Recorrido por issues y PRs (3 min):** muestra un issue que le asignaron y el PR correspondiente.
4. **Preguntas del docente (3-5 min):** sobre su módulo, decisiones técnicas y aprendizaje.
5. **Registro:** cada objetivo se marca como logrado / parcial / no logrado en la planilla.

5. Alcance

Incluido: todo el temario del curso aplicado al trabajo integrador (entrada validada, condicionales, bucles, funciones, colecciones, menú en memoria, métodos de texto), flujo profesional de Git (README, issues, ramas, pull requests, main protegida).

Objetivo mínimo: demostrar comprensión del programa integrador del grupo, explicar el propio rol en el equipo y realizar una modificación guiada simple sobre el código.

6. Verificación práctica (versiones equivalentes)

La verificación es una modificación guiada sobre el propio trabajo integrador del grupo. No se escribe un programa nuevo. Puntaje total: 100 puntos.

- **Parte 1 — Modificación guiada (60 pts):** agregar una funcionalidad simple al programa integrador (p. ej., agregar un nuevo producto y mostrarlo, o agregar una validación faltante).
- **Parte 2 — Explicación (20 pts):** responder por escrito cómo funciona la modificación y dónde se inserta en el programa.
- **Parte 3 — Defensa individual (20 pts):** repositorio en orden, demo funcional, respuestas a preguntas del docente.

7. Criterios de calificación

Componente	Puntaje
Parte 1 — Modificación funcionando	60 pts
Parte 2 — Explicación escrita	20 pts
Parte 3 — Defensa individual	20 pts

8. Condiciones de resolución

Individual. Cada estudiante trabaja sobre su propia copia del repositorio grupal. La modificación se entrega en una rama nueva (`evaluacion-u4`) con su commit.

9. Regla de equivalencia entre versiones

Las versiones A y B comparten la misma estructura de verificación: misma modificación guiada (agregar un producto/película), misma exigencia de explicación. Cambian los tokens de dominio que aparecen en la consigna de la parte práctica.

10. Asignación de versiones

Fila impar → A (verdulería). Fila par → B (videoteca). Se registra en la planilla.

11. Devolución

Encuentro 33. Estudiantes con menos de 60 pts en la verificación o defensa insuficiente coordinan recuperación con el docente.

Evaluación de la Unidad 4 — Encuentro 32 — Versión A

Dominio de esta versión: **verdulería** (los tokens de ejemplo usan el dominio del TP grupal). Duración: 40 minutos (ventana de verificación práctica). Puntaje total: 100 puntos (60 modificación + 20 explicación + 20 defensa). Resolución individual.

Antes de empezar

- Trabajá sobre **tu copia local** del repositorio grupal del trabajo integrador.
- No crees una carpeta nueva: trabajá en el mismo `main.py` o en un archivo nuevo dentro del proyecto.
- Creá una rama nueva: `git switch -c evaluacion-u4` y al terminar hacé commit en esa rama (sin mergear a main).
- Seguí las convenciones del curso.

Objetivos de la verificación

1. Demostrar que comprendés el código del trabajo integrador de tu grupo.
2. Realizar una modificación guiada sobre el programa existente.
3. Explicar por escrito los cambios realizados.

Material provisto

Tu propio trabajo integrador del grupo en el estado en que quedó al cierre del encuentro 31.

Parte 1 — Modificación guiada (60 puntos)

Agregá una **nueva funcionalidad** al programa integrador de tu grupo. La funcionalidad es la misma para todas las versiones, los ejemplos usan tokens del dominio verdulería:

Consigna: agregá una opción en el menú principal que permita al usuario **buscar un producto por nombre y mostrar su información completa** (nombre, `productPrice`, `stockWeight`). Si no existe, mostrar “Producto no encontrado.”

Ítem	Consigna	Pts
1a	Agregar la opción en el menú principal del programa (ej. “5. Buscar producto”)	10
1b	Implementar la función <code>find_product(name)</code> que recorre <code>productList</code> y devuelve el producto si existe, o <code>None</code> si no	20
1c	Mostrar la información completa del producto (nombre, precio, kilos/copias disponibles) con f-strings	15
1d	Validar que el nombre de búsqueda no esté vacío	15
Total Parte 1		60

Parte 2 — Explicación escrita (20 puntos)

Agregá un comentario multilínea al final del archivo (o en un archivo `EXPLICACION.md` dentro de la rama) respondiendo:

Ítem	Pregunta	Pts
2a	¿Qué archivo(s) modificaste y qué función agregaste?	10
2b	¿Cómo verificaste que la modificación funciona?	10
Total Parte 2		20

Parte 3 — Defensa individual (20 puntos)

Durante la defensa con el docente se evalúa:

Ítem	Qué se observa	Pts
3a	El repositorio tiene README, issues cerrados, PR mergeados, main protegida	10
3b	El estudiante ejecuta el programa, explica su módulo y responde preguntas	10
Total Parte 3		20

Batería de verificación

Prueba	Entrada	Salida esperada
Opción 5	5	Pide nombre del producto
Buscar producto existente	papa	Muestra nombre, precio y kilos
Buscar producto inexistente	xyz	"Producto no encontrado."
Buscar con nombre vacío	(enter)	"El nombre no puede estar vacío."

Al terminar

```
git add .
git commit -m "evaluacion-u4: agregar busqueda de producto"
# NO hacer push a main. Solo dejar el commit en la rama evaluacion-u4.
# EL docente revisa la rama durante la defensa.
```

Evaluación de la Unidad 4 — Encuentro 32 — Versión A

Dominio de esta versión: **videoteca** (los tokens de ejemplo usan el dominio del TP grupal). Duración: 40 minutos (ventana de verificación práctica). Puntaje total: 100 puntos (60 modificación + 20 explicación + 20 defensa). Resolución individual.

Antes de empezar

- Trabajá sobre **tu copia local** del repositorio grupal del trabajo integrador.
- No crees una carpeta nueva: trabajá en el mismo `main.py` o en un archivo nuevo dentro del proyecto.
- Creá una rama nueva: `git switch -c evaluacion-u4` y al terminar hacé commit en esa rama (sin mergear a main).
- Seguí las convenciones del curso.

Objetivos de la verificación

1. Demostrar que comprendés el código del trabajo integrador de tu grupo.
2. Realizar una modificación guiada sobre el programa existente.
3. Explicar por escrito los cambios realizados.

Material provisto

Tu propio trabajo integrador del grupo en el estado en que quedó al cierre del encuentro 31.

Parte 1 — Modificación guiada (60 puntos)

Agregá una **nueva funcionalidad** al programa integrador de tu grupo. La funcionalidad es la misma para todas las versiones, los ejemplos usan tokens del dominio videoteca:

Consigna: agregá una opción en el menú principal que permita al usuario **buscar un película por nombre y mostrar su información completa** (nombre, `moviePrice`, `stockCopies`). Si no existe, mostrar “Película no encontrado.”

Ítem	Consigna	Pts
1a	Agregar la opción en el menú principal del programa (ej. “5. Buscar película”)	10
1b	Implementar la función <code>find_product(name)</code> que recorre <code>movieList</code> y devuelve el película si existe, o <code>None</code> si no	20
1c	Mostrar la información completa del película (nombre, tarifa, copias/copias disponibles) con f-strings	15
1d	Validar que el nombre de búsqueda no esté vacío	15
Total Parte 1		60

Parte 2 — Explicación escrita (20 puntos)

Agregá un comentario multilínea al final del archivo (o en un archivo `EXPLICACION.md` dentro de la rama) respondiendo:

Ítem	Pregunta	Pts
2a	¿Qué archivo(s) modificaste y qué función agregaste?	10
2b	¿Cómo verificaste que la modificación funciona?	10
Total Parte 2		20

Parte 3 — Defensa individual (20 puntos)

Durante la defensa con el docente se evalúa:

Ítem	Qué se observa	Pts
3a	El repositorio tiene README, issues cerrados, PR mergeados, main protegida	10
3b	El estudiante ejecuta el programa, explica su módulo y responde preguntas	10
Total Parte 3		20

Batería de verificación

Prueba	Entrada	Salida esperada
Opción 5	5	Pide nombre del película
Buscar película existente	papa	Muestra nombre, tarifa y copias
Buscar película inexistente	xyz	"Película no encontrado."
Buscar con nombre vacío	(enter)	"El nombre no puede estar vacío."

Al terminar

```
git add .
git commit -m "evaluacion-u4: agregar busqueda de película"
# NO hacer push a main. Solo dejar el commit en la rama evaluacion-u4.
# EL docente revisa la rama durante la defensa.
```